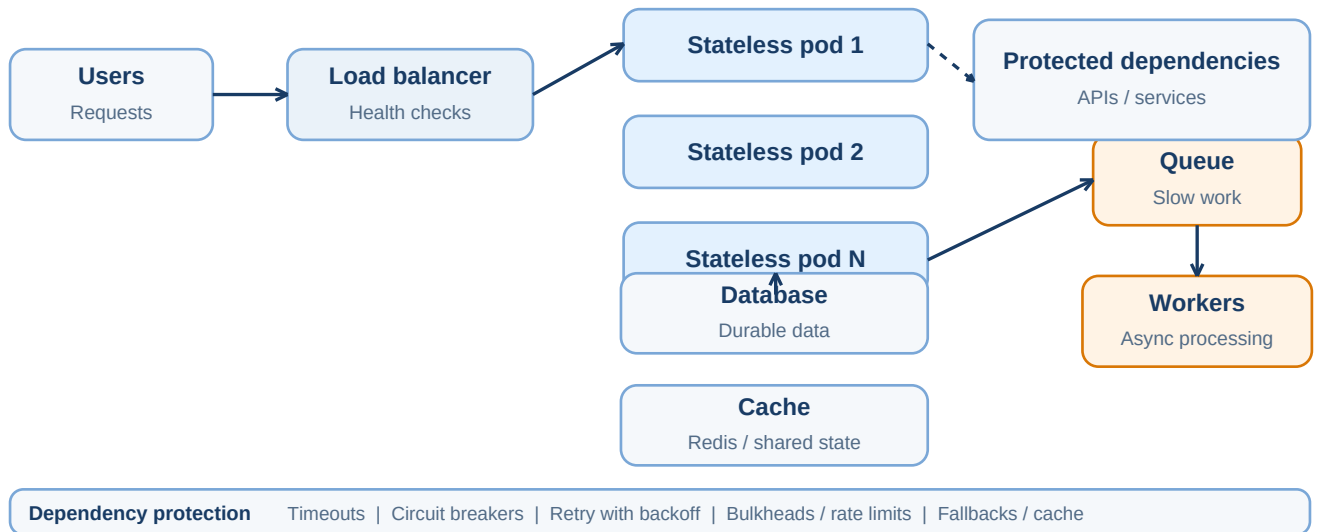


Scalable Application Architecture

A high-level guide for applications handling very large request volumes

Core request flow



The system stays reliable by keeping the application stateless, adding capacity horizontally, moving slow work to background processing, and preventing one dependency from taking down everything else.

1. Stateless application

Do not keep user session or important request state only in one server's memory. Store shared state in services such as Redis, a database, or object storage. Any healthy application instance can then handle any request.

2. Horizontally scalable

Add more copies of the application instead of making one server increasingly large. A load balancer distributes incoming requests across the healthy copies.

3. Synchronous and asynchronous work

Keep the user-facing request path short. Use synchronous calls only when the user needs an immediate result. Put non-urgent work - emails, notifications, reports, image processing, and retries - on a queue for background workers.

4. Dependency protection

A dependency is anything your application relies on: a database, cache, payment provider, email/SMS service, another microservice, or third-party API. Dependency protection prevents a slow or failing dependency from causing a full application outage.

Dependency protection safeguards

Safeguard	Why it matters	Example
Timeout	Stop waiting after a safe limit.	End a payment call after 2 seconds.
Circuit breaker	Pause calls to a repeatedly failing dependency.	Temporarily stop calling a failed payment API.
Retry with backoff	Retry transient failures slowly, without creating a traffic spike.	Retry after 1s, 2s, then 4s.
Bulkhead / rate limit	Reserve resources so one dependency cannot consume everything.	Allow only 100 concurrent calls to a service.
Queue	Move non-urgent work outside the request path.	Send email from a background worker.
Fallback / cache	Return a reduced but safe result during failure.	Show cached recommendations if the service fails.

Example: payment dependency failure

Without protection, thousands of requests may wait for a slow payment provider. Application threads and connections become exhausted, which can make even unrelated pages fail.

With protection: apply a short timeout; retry only safe temporary failures; open the circuit breaker after repeated errors; and return a clear 'payment pending - please retry' response. The rest of the application remains available.

Key principle

A dependency must be allowed to fail locally and gracefully, rather than spreading failure across the entire system.